

1)

Gerando com o programa no apêndice $N = 10.000$ dados x com distribuição gaussiana de valor verdadeiro $x_0 = 50$ e desvio padrão verdadeiro $\sigma_0 = 15$ podemos tirar os valores estatísticos dos dados, como:

$$x_m = 49.993$$

$$\sigma_x = 14.985$$

Entre o intervalo $x_0 - 1 \sigma_0 < x < x_0 + 1 \sigma_0 = 6844$ dados e Frequência relativa = 0.68440

Entre o intervalo $x_0 - 2 \sigma_0 < x < x_0 + 2 \sigma_0 = 9553$ dados e Frequência relativa = 0.95530

Entre o intervalo $x_0 - 3 \sigma_0 < x < x_0 + 3 \sigma_0 = 9966$ dados e Frequência relativa = 0.99660

Estes valores condizem com a frequência relativa esperada para a FDP normal e sua média e desvio padrão se aproximam do valor tomado antes da simulação.

Os dados foram simulados 1000 vezes para se obter aproximações dos valores estatísticos por Monte-Carlo.

```
[5.00005838e+01 1.49996790e+01 6.82574700e+03 6.82574700e-01
9.54501400e+03 9.54501400e-01 9.97287000e+03 9.97287000e-01]
[1.48244874e-01 1.05018338e-01 4.68616795e+01 4.68616795e-03
1.97298015e+01 1.97298015e-03 5.10010893e+00 5.10010893e-04]
```

$$x_m = 50.00(15)$$

$$\sigma_x = 15.00(11)$$

Entre o intervalo $x_0 - 1 \sigma_0 < x < x_0 + 1 \sigma_0 = 6826(46)$ dados e Frequência relativa = 0.6826(46)

Entre o intervalo $x_0 - 2 \sigma_0 < x < x_0 + 2 \sigma_0 = 9545(20)$ dados e Frequência relativa = 0.9545(20)

Entre o intervalo $x_0 - 3 \sigma_0 < x < x_0 + 3 \sigma_0 = 9972.9(51)$ dados e Frequência relativa = 0.99729(51)

Ao gerar o mesmo número de dados por algoritmo bootstrap:

$$X_m = 49.910$$

$$S_x = 15.487$$

intervalo $x_0 - 1 \sigma_0 < x < x_0 + 1 \sigma_0 = 6735$ e Frequência relativa = 0.67350

intervalo $x_0 - 2 \sigma_0 < x < x_0 + 2 \sigma_0 = 9515$ e Frequência relativa = 0.95150

intervalo $x_0 - 3 \sigma_0 < x < x_0 + 3 \sigma_0 = 9962$ e Frequência relativa = 0.99620

Desvio padrão da média

MC	BootStrap
0.48	0.49

Ssx

MC	BootStrap
0.34	0.35

No geral os valores de bootstrap se aproximam do método MC, entretanto o método MC continua razoavelmente melhor

2)

Tomando a seguinte função como uma F.D.P:

$$f(x) = \begin{cases} A(1 - |x|^g) & \text{se } |x| \leq 1 \\ 0 & \text{caso contrário} \end{cases}$$

A função é uma F.D.P e simétrica em relação ao eixo y, logo:

$$A \int_0^1 (1-x)^g dx = A \int_{-1}^0 (1+x)^g dx = \frac{1}{2}$$

$$A \int_0^1 (1-x)^g dx = \frac{1}{2}$$

$$= A \int (1-x)^g dx$$

Usando $u = 1 - x$ e $du = -dx$:

$$\left| = -A \int u^g du \right.$$

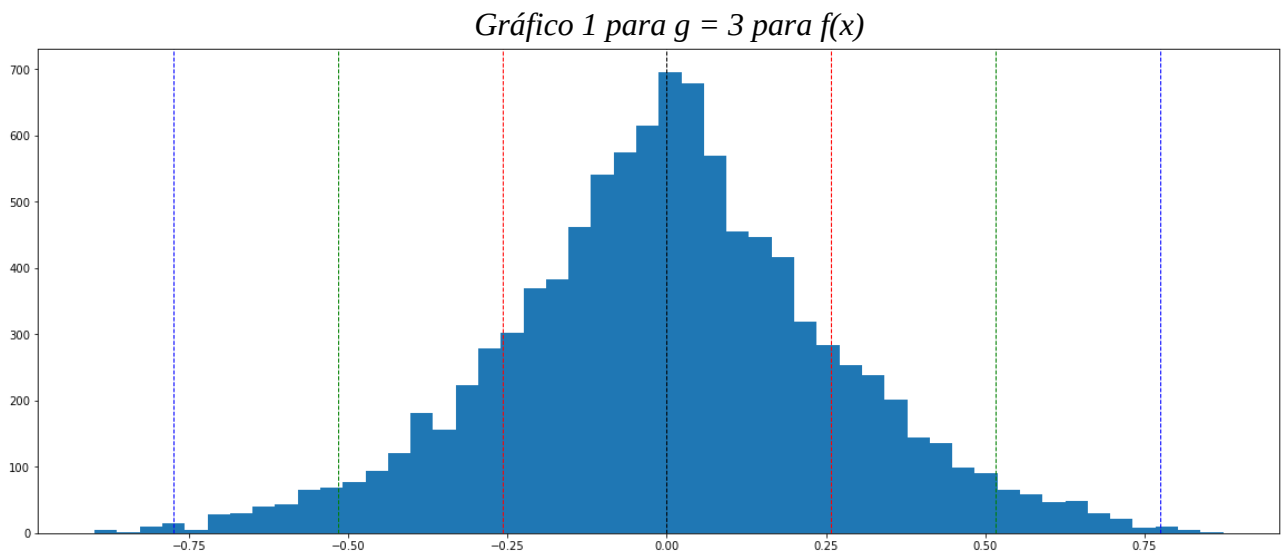
$$= -\frac{A(1-x)^{g+1}}{g+1}$$

$$A = \frac{g+1}{2}$$

Para o valor esperado x_0 , será $x_0=0$ por se tratar de uma função simétrica em relação ao eixo y.

$$\langle \phi(x) \rangle = \int_{-\infty}^{+\infty} \phi(x) f(x) dx = 0$$

Para $g = 3$



*Múltiplos [1,2,3] do desvio padrão coloridos
FDP obtida por método da exclusão*

$$A = \frac{g+1}{2}$$

$$A = 2$$

Para o valor esperado, por se tratar de uma função simétrica no eixo y:

$$\langle \phi(x) \rangle = \int_{-\infty}^{+\infty} \phi(x) f(x) dx$$

$$2 \int_{-1}^1 x (1 - |x|)^3 dx = 0$$

Para o desvio padrão:

$$\sigma_0^2 = \langle (x - x_0)^2 \rangle = \int_{-\infty}^{+\infty} (x - x_0)^2 f(x) dx$$

Logo:

$$\sqrt{2 \int_{-1}^1 x^2 (1 - |x|)^3 dx}$$

Para:

$$P(x \in [x_0 - \sigma_0, x_0 + \sigma_0])$$

e considerando a simetria da função pelo eixo y

$$2 \int_{-\frac{\sqrt{15}}{15}}^{\frac{\sqrt{15}}{15}} (1 - |x|)^3 dx = 4 \int_0^{\frac{\sqrt{15}}{15}} (1 - x)^3 dx$$

$$4 \int (1 - x)^3 dx = \underline{-(x - 1)^4} + c$$

$$4 \int_0^{\frac{1}{\sqrt{15}}} (1 - x)^3 dx = 4 \left(\frac{16}{15 \sqrt{15}} - \frac{91}{900} \right) \approx \underline{0.6972}$$

Numericamente pelo programa no apêndice foi obtido 6993 dados e frequência relativa 0.6993

Para:

$$P(x \in [x_0 - 2\sigma_0, x_0 + 2\sigma_0])$$

e considerando a simetria da função pelo eixo y

$$4 \int (1 - x)^3 dx = \underline{-(x - 1)^4} + c$$

$$4 \int_0^{\frac{2}{\sqrt{15}}} (1 - x)^3 dx = 4 \left(\frac{38}{15 \sqrt{15}} - \frac{94}{225} \right) \approx 0.9453$$

Numericamente pelo programa no apêndice foi obtido 9412 dados e frequência relativa 0.9412

Para:

$$P(x \in [x_0 - 3\sigma_0, x_0 + 3\sigma_0])$$

e considerando a simetria da função pelo eixo y

$$4 \int (1 - x)^3 dx = \underline{-(x - 1)^4} + c$$

$$4 \int_0^{\sqrt{\frac{3}{5}}} (1 - x)^3 dx = 4 \left(\frac{8 \sqrt{\frac{3}{5}}}{5} - \frac{99}{100} \right) \approx 0.997$$

Numericamente pelo programa no apêndice foi obtido 9970 dados e frequência relativa 0.997

3)

Tomando a função:

$$f(x) = \begin{cases} \frac{3}{2}x^2 & \text{se } |x| \leq 1 \\ 0 & \text{caso contrário} \end{cases}$$

Para o valor esperado

$$\langle \phi(x) \rangle = \int_{-\infty}^{+\infty} \phi(x) f(x) dx$$

$$\int_{-1}^1 \frac{3x^3}{2} dx = 0$$

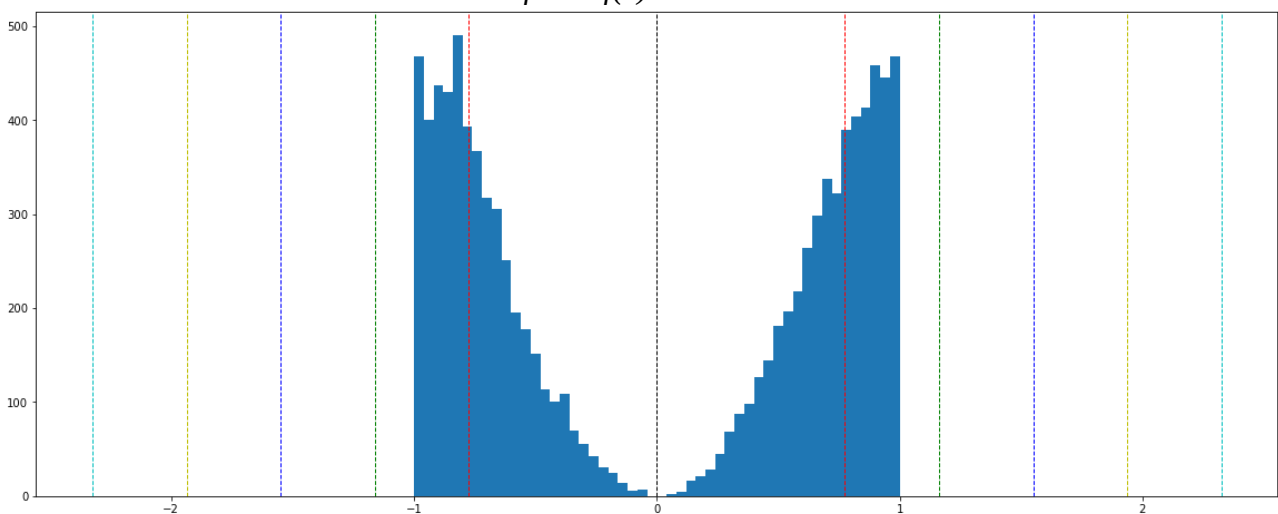
Para o desvio padrão:

$$\sigma_0^2 = \langle (x - x_0)^2 \rangle = \int_{-\infty}^{+\infty} (x - x_0)^2 f(x) dx$$

logo:

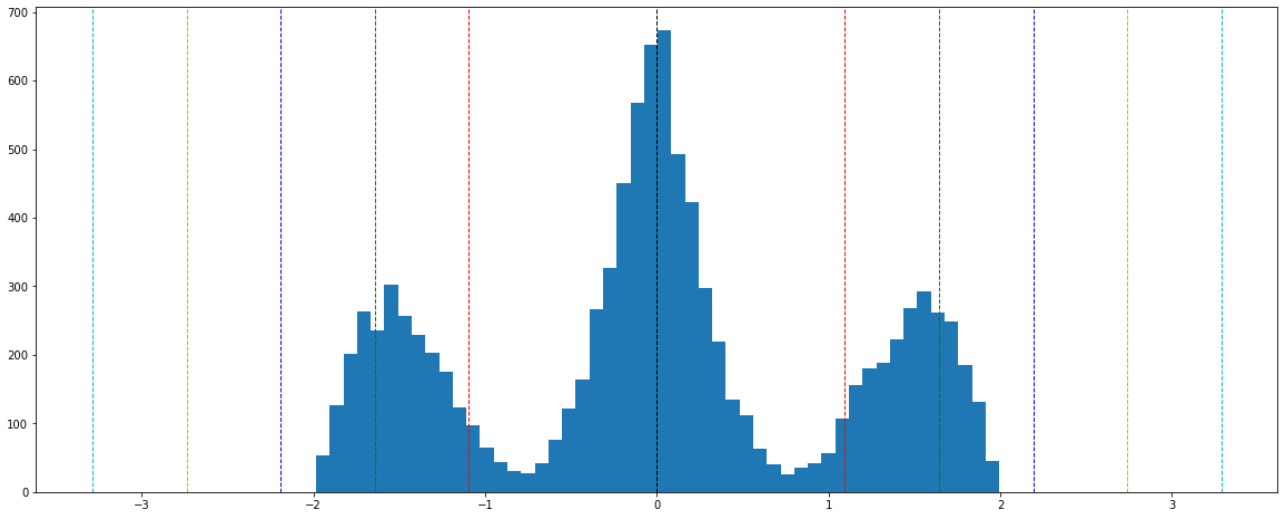
$$\sigma_0 = \sqrt{\int_{-1}^1 \frac{3x^4}{2} dx} = \sqrt{\frac{3}{5}}$$

Gráfico 2 $f(x)$ com $M=1$



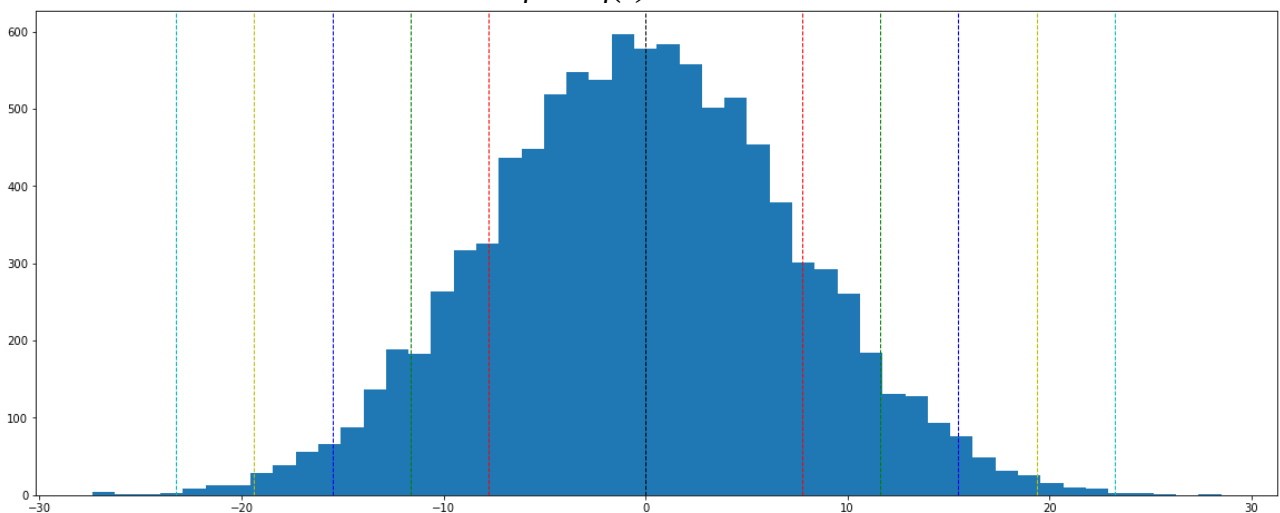
Múltiplos $[1,2,3]$ do desvio padrão coloridos
FDP obtida por método da exclusão

Gráfico 3 $f(x)$ com $M=2$



Múltiplos $[1,2,3]$ do desvio padrão coloridos
FDP obtida por método da exclusão

Gráfico 4 $f(x)$ com $M=100$



Múltiplos $[1,2,3]$ do desvio padrão coloridos
FDP obtida por método da exclusão

Observando os gráficos 2,3,4, constata-se que M tendendo ao infinito se valida o teorema central do limite e a função aproxima-se de uma FDP normal. Tende sua curtose para a curtose = 3 da FDP normal quando M tende ao infinito

Quadro 3.1 – Respostas do Exercício 3.

	(1)	(2)	(3) M=3	(4) M=5	(5) M=10	(6) M=100
λ	$n(x - x_0 \leq \lambda \sigma_x)$	$n(y - y_0 \leq \lambda \sigma_y)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$
1	5104	5595	7304	6675	6939	6925
1,5	10000	8599	8168	8793	8807	8748
2	10000	10000	9851	9539	9626	9601
2,5	10000	10000	10000	9966	9907	9911
3	10000	10000	10000	10000	9987	9988
A	-0.01	0.011	-0.00	0.02	-0.00	0.02
K	-1.80	-0.85	-0.59	-0.32	-0.12	-0.07

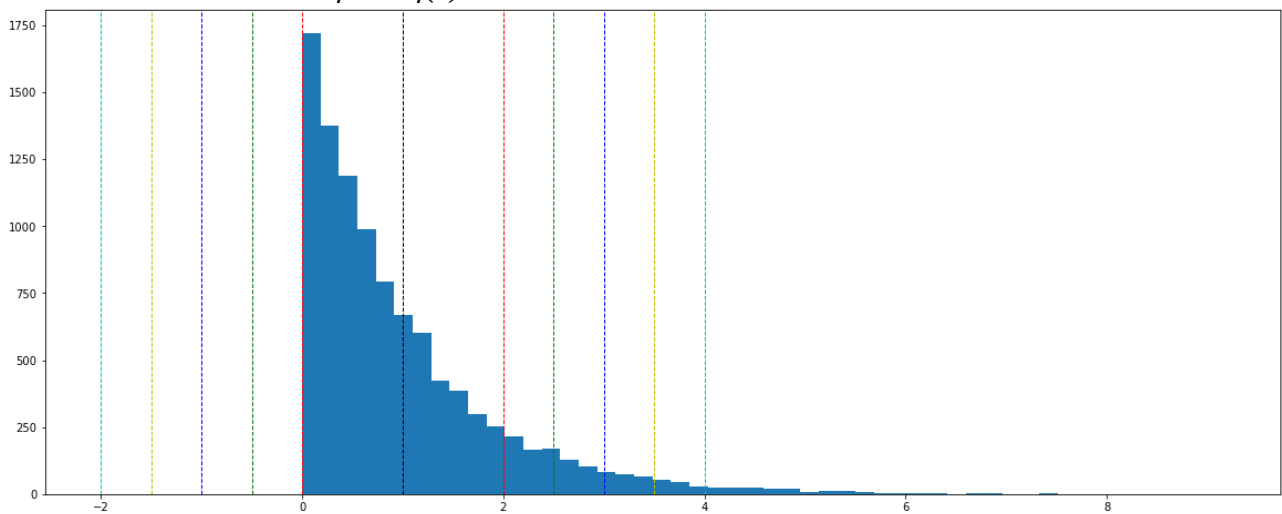
Resultados dos parâmetros após simulação de M somas

4)

Para

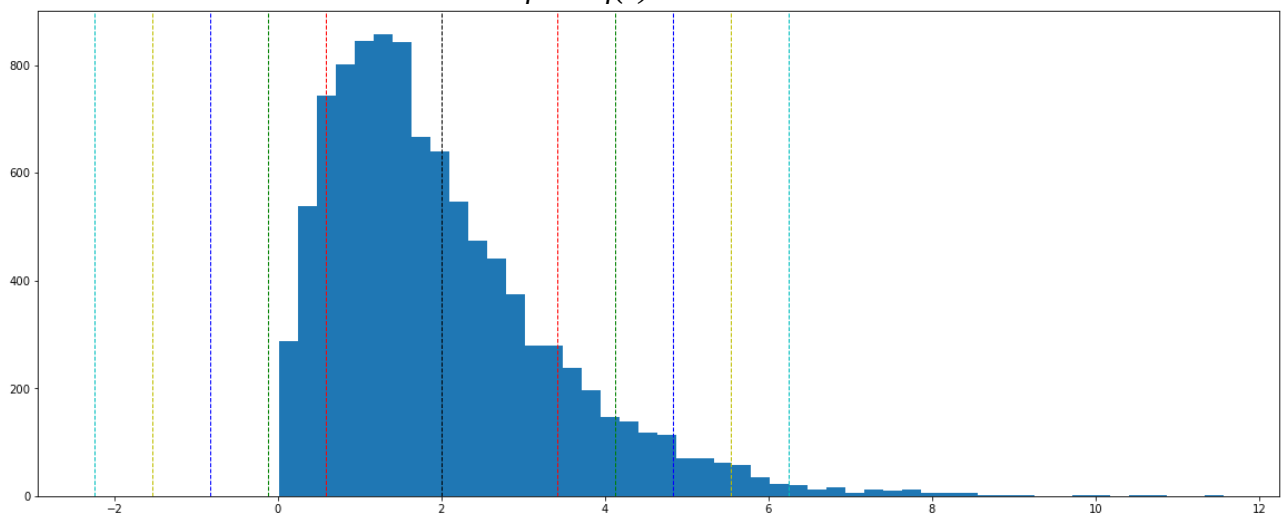
$$f(x) = \begin{cases} e^{-x} & \text{se } x \geq 0 \\ 0 & \text{caso contrário} \end{cases}$$

Gráfico 5 $f(x)$ com $M=1$



*Múltiplos [1,2,3] do desvio padrão coloridos
FDP obtida por método da inversão*

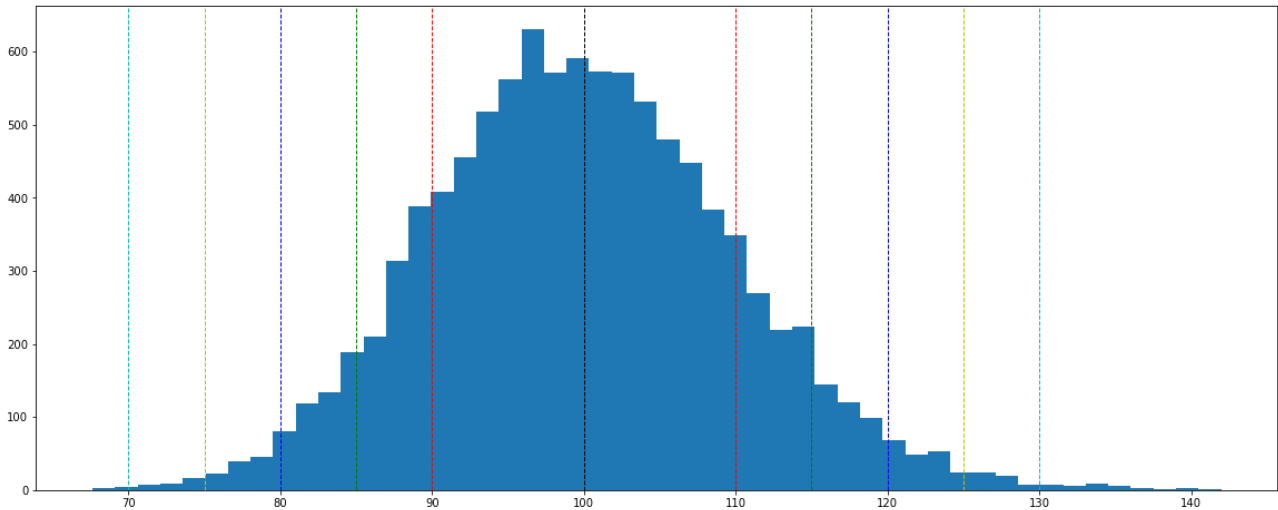
Gráfico 6 $f(x)$ com $M=2$



*Múltiplos [1,2,3] do desvio padrão coloridos
FDP obtida por método da inversão*

*Observando os gráficos 4,5,6, constata-se que M tendendo ao infinito se valida o teorema central do limite e a função aproxima-se de uma FDP normal.
Diminui sua assimetria e no infinito tende à mesma kurtose da FDP normal*

Gráfico 7 $f(x)$ com $M=100$



Múltiplos $[1,2,3]$ do desvio padrão coloridos
FDP obtida por método da inversão

Quadro 3.2 – Respostas do Exercício 4

	(1)	(2)	(3) $M=3$	(4) $M=5$	(5) $M=10$	(6) $M=100$
λ	$n(x - x_0 \leq \lambda \sigma_x)$	$n(y - y_0 \leq \lambda \sigma_y)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$	$n(S - S_0 \leq \lambda \sigma_S)$
1	8679	7401	7178	6990	6935	6871
1,5	9184	9165	9081	8935	8803	8733
2	9500	9549	9565	9577	9591	9570
2,5	9701	9766	9763	9808	9838	9865
3	9823	9881	9873	9918	9938	9963
A	1.91	1.34	1.17	0.86	0.60	0.22
K	5.00	2.51	2.12	1.06	0.56	0.15

Resultados dos parâmetros após simulação de M somas

A proposta da simulação foi demonstrar como válido o teorema central do limite.

A soma de muitos erros aleatórios numa função densidade de probabilidade não gaussiana tenderá a uma gaussiana conforme uma somatória infinita de erros.

O número de dados entre os intervalos 1, 2 e 3 desvio padrão se aproximam aos valores esperados para uma função gaussiana conforme M aumenta.

Para a primeira função é notável como a curtose começa a aumentar (quadro 3.1) e para a segunda função é notável como a assimetria começa a diminuir (quadro 3.2) em uma função exponencial muito assimétrica.

1

```
In [1]: from matplotlib import pyplot as plt
import numpy as np
import scipy.stats as stats
plt.rcParams['figure.figsize'] = [20, 8]
nU=10300282
```

```
In [2]: np.random.seed(nU)
N=10000
x= 50+15*np.random.randn(N)
ss=[sum(50<x+15*i)-sum(50<x-15*i) for i in range(1,4)]
print('Xm = %.3f'
      %np.mean(x), '\nSx = %.3f'
      %np.std(x, ddof=1),
      )
for i in ss:
    print('intervalo x0 +-%d' %ss.index(i), '*s0 = %.f e' %i, 'Frequência relativa =
```

```
Xm = 49.993
Sx = 14.985
intervalo x0 +-0 *s0 = 6844 e Frequência relativa = 0.68440
intervalo x0 +-1 *s0 = 9553 e Frequência relativa = 0.95530
intervalo x0 +-2 *s0 = 9966 e Frequência relativa = 0.99660
```

```
In [3]: NN=1000
ss=[]
xm=[]
Fr=[]
np.random.seed(nU)
R=[]
for i in range(NN):
    N=10000
    x= 50+15*np.random.randn(N)
    ss=[sum(50<x+15*i)-sum(50<x-15*i) for i in range(1,4)]
    R.append([
        np.mean(x),
        np.std(x, ddof=1),
        ss[0], ss[0]/N, ss[1], ss[1]/N, ss[2], ss[2]/N,
    ])
print(np.mean(R, axis=0))
print(np.std(R, axis=0, ddof=1))

[5.00005838e+01  1.49996790e+01  6.82574700e+03  6.82574700e-01
 9.54501400e+03  9.54501400e-01  9.97287000e+03  9.97287000e-01]
[1.48244874e-01  1.05018338e-01  4.68616795e+01  4.68616795e-03
 1.97298015e+01  1.97298015e-03  5.10010893e+00  5.10010893e-04]
```

```
In [20]: def geraConjuntoBootstrap( x ):
    N = len( np.asarray(x) )
    xB = []
    INDs= np.random.randint( 0, N, size=N)
    for ind in INDs:
        xB.append( x[ind] )
    return xB
```

```
In [21]: np.random.seed(nU)
```

```
xB=geraConjuntoBootstrap( x )
```

In [24]:

```
ss=[sum(50<x+15*i)-sum(50<x-15*i) for i in range(1,4)]
print('Xm = %.3f'
      %np.mean(xB),'\nSx = %.3f'
      %np.std(xB,ddof=1),
      )
for i in ss:
    print('intervalo x0 +-%d' %ss.index(i),'*s0 = %.f e' %i,'Frequência relativa =
```

Xm = 49.910

Sx = 15.487

intervalo x0 +-0 *s0 = 6735 e Frequência relativa = 0.67350

intervalo x0 +-1 *s0 = 9515 e Frequência relativa = 0.95150

intervalo x0 +-2 *s0 = 9962 e Frequência relativa = 0.99620

In [31]:

```
print('Desvio padrão da média')
print('MC\t\t\t\tBootStrap')
print(np.std(x)/NN**.5,'\t\t',np.std(xB)/NN**.5)
print('\nSsx')
print('MC\t\t\t\tBootStrap')
print(np.std(x)/(2*(NN-1))**.5,'\t\t',np.std(xB)/(2*(NN-1))**.5)
ss=[sum(50<x+15*i)-sum(50<x-15*i) for i in range(1,4)]
ssb=[sum(50<np.array(xB)+15*i)-sum(50<np.array(xB)-15*i) for i in range(1,4)]
for i in ss:
    print('\nSn')
    print('MC')
    print('intervalo x0 +-%d' %ss.index(i),'*s0 = %.f e' %i,'Frequência relativa =
    print('BootStrap')
    print('intervalo x0 +-%d' %ss.index(i),'*s0 = %.f e' %i,'Frequência relativa =
    print('\nSf')
    print('MC')
    print('intervalo x0 +-%d' %ss.index(i),'*s0 = %.f e' %i,'Frequência relativa =
    print('BootStrap')
    print('intervalo x0 +-%d' %ss.index(i),'*s0 = %.f e' %i,'Frequência relativa =
```

Desvio padrão da média

MC

0.48226641593504077

BootStrap

0.4897284268736163

Ssx

MC

0.3411844879595775

BootStrap

0.34646356669511524

Sn

MC

intervalo x0 +-0 *s0 = 6735 e Frequência relativa = 67.35000

BootStrap

intervalo x0 +-0 *s0 = 6735 e Frequência relativa = 67.35000

Sf

MC

intervalo x0 +-0 *s0 = 6735 e Frequência relativa = 0.02130

BootStrap

intervalo x0 +-0 *s0 = 6735 e Frequência relativa = 0.02130

Sn

MC

intervalo x0 +-1 *s0 = 9515 e Frequência relativa = 95.15000

BootStrap

intervalo x0 +-1 *s0 = 9515 e Frequência relativa = 95.15000

```

Sf
MC
intervalo x0 +-1 *s0 = 9515 e Frequência relativa = 0.03009
BootStrap
intervalo x0 +-1 *s0 = 9515 e Frequência relativa = 0.03009

Sn
MC
intervalo x0 +-2 *s0 = 9962 e Frequência relativa = 99.62000
BootStrap
intervalo x0 +-2 *s0 = 9962 e Frequência relativa = 99.62000

Sf
MC
intervalo x0 +-2 *s0 = 9962 e Frequência relativa = 0.03150
BootStrap
intervalo x0 +-2 *s0 = 9962 e Frequência relativa = 0.03150

 $F_X(x) = P(X \leq x) = P(-\infty \leq X \leq x) = 1 - e^{-x}$   $x = -\ln(1-y)$ 

```

2

In [7]:

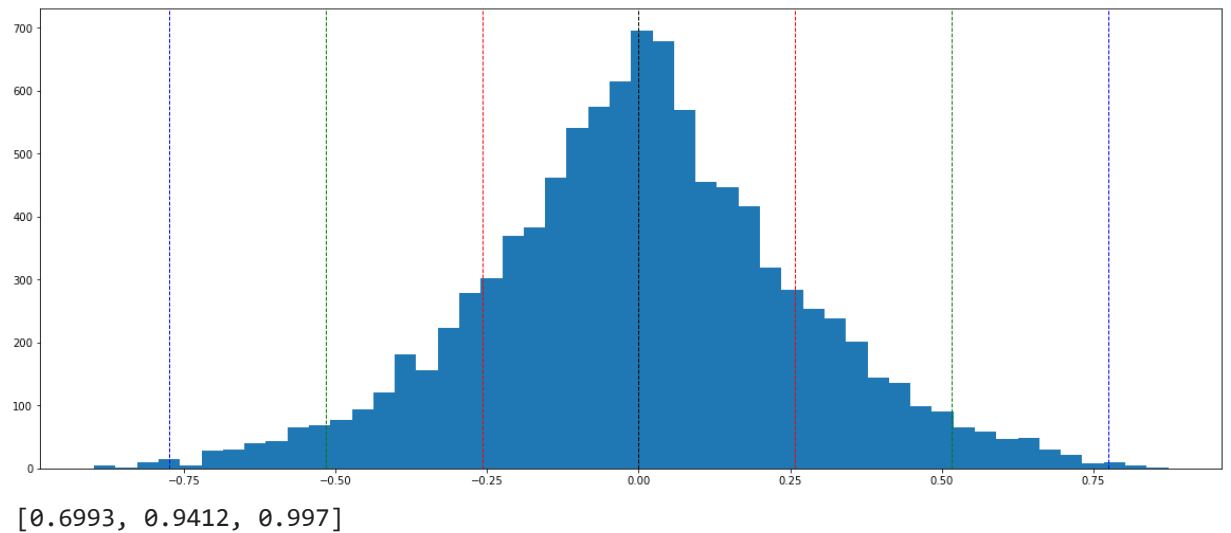
```

from matplotlib import pyplot as plt
import numpy as np
import scipy.stats as stats
plt.rcParams['figure.figsize'] = [20, 8]
nU=10300282
def geraDadosExclusao( N=1, f=lambda x:2*(1-abs(x))**3, xI=-1, xS=+1, yMax=2 ):
    x = []
    while( len(x)<N ):
        xc = np.random.rand()*(xS-xI) + xI
        yv = yMax * np.random.rand()
        if yv<=f(xc):
            x.append(xc)
    return x

N=10000
np.random.seed(nU)
dados =np.array(geraDadosExclusao(N))
lamb=np.arange(1,4)
sx=15**.5/15
x0=0
r=[]
colors=['r','g','b','y','c']

plt.hist(dados,bins=50)
plt.axvline(x0, color='k', linestyle='dashed', linewidth=1)
for iii in lamb:
    r.append((sum(dados<((iii*sx)+x0))-sum(dados<(-(iii*sx)+x0)))/N)
    plt.axvline(x0+(iii*sx), color=colors[int(np.where(lamb==iii)[0])], linestyle='d')
    plt.axvline(x0-(iii*sx), color=colors[int(np.where(lamb==iii)[0])], linestyle='d')
plt.show()
print(r)

```

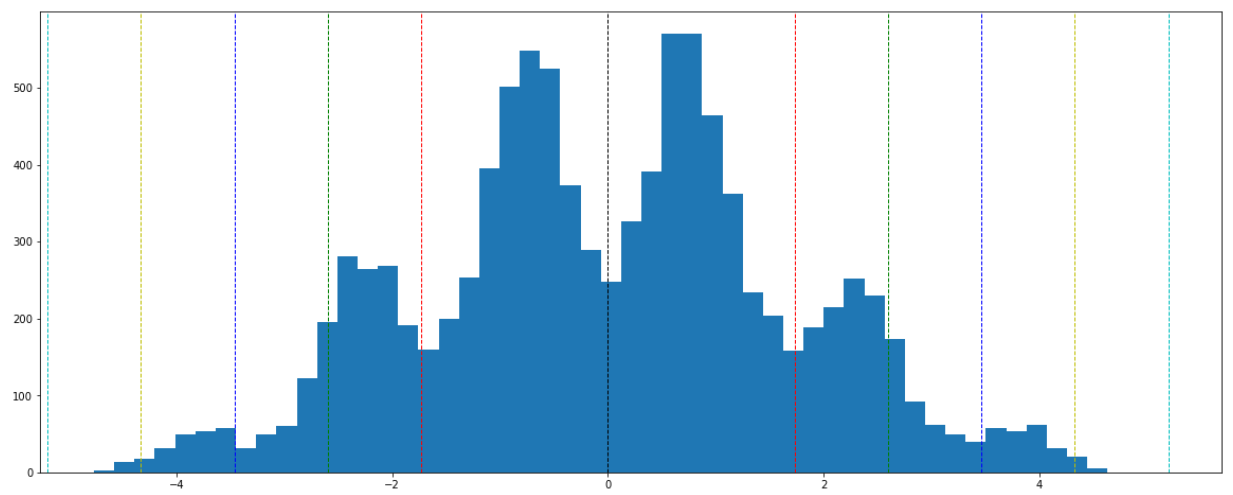
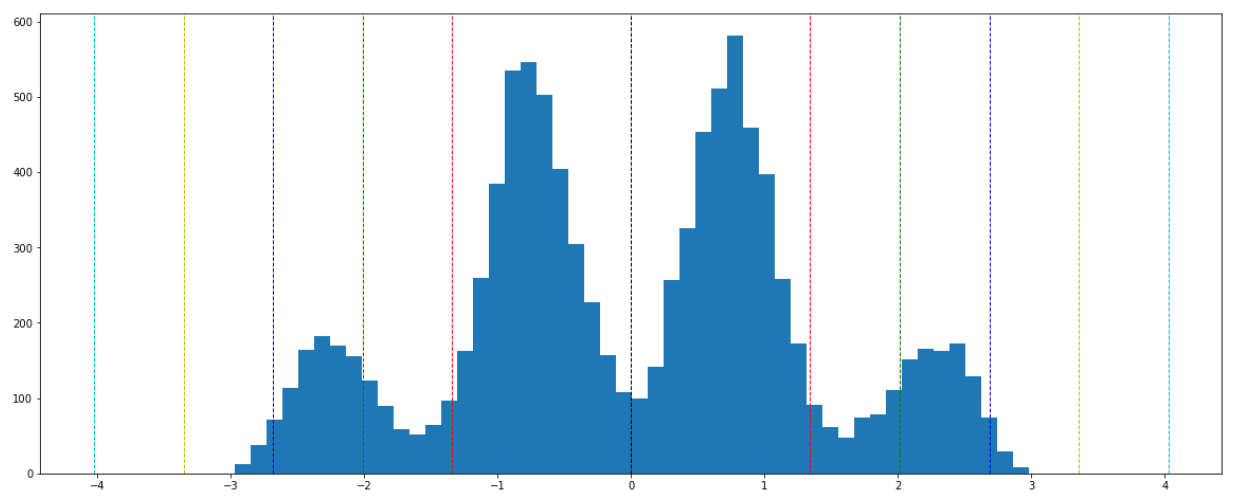
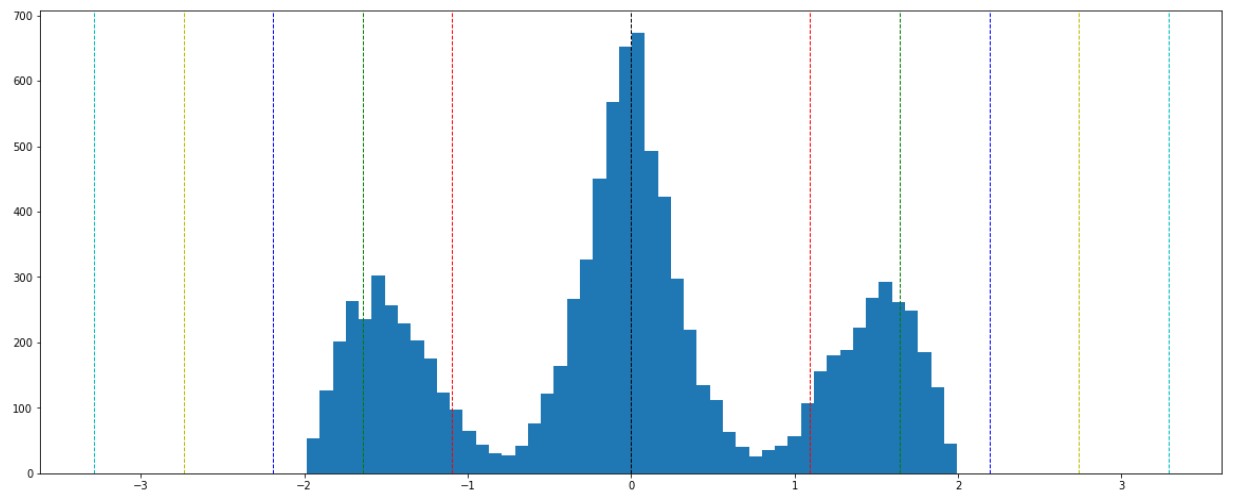
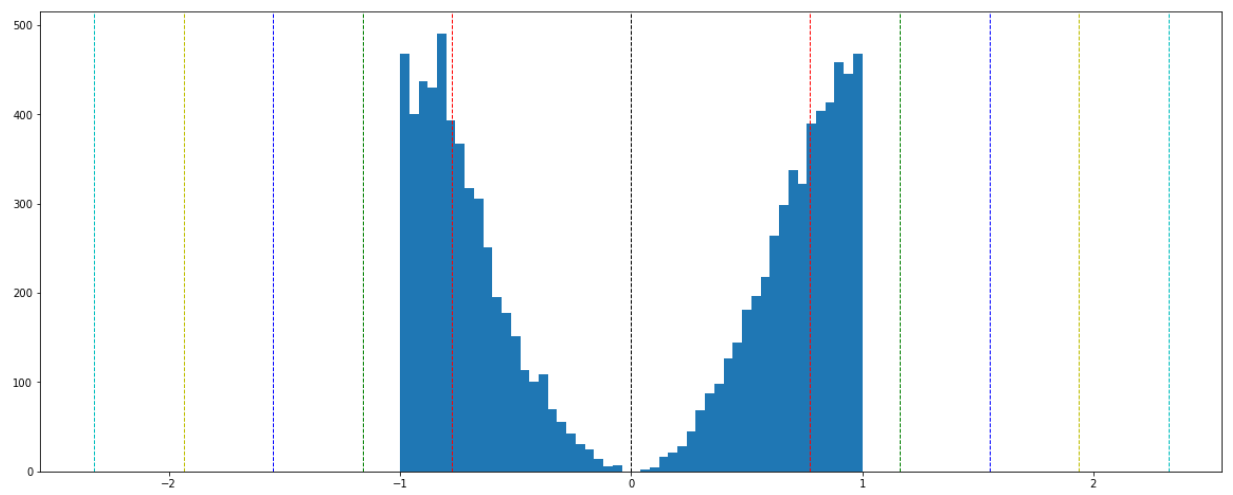


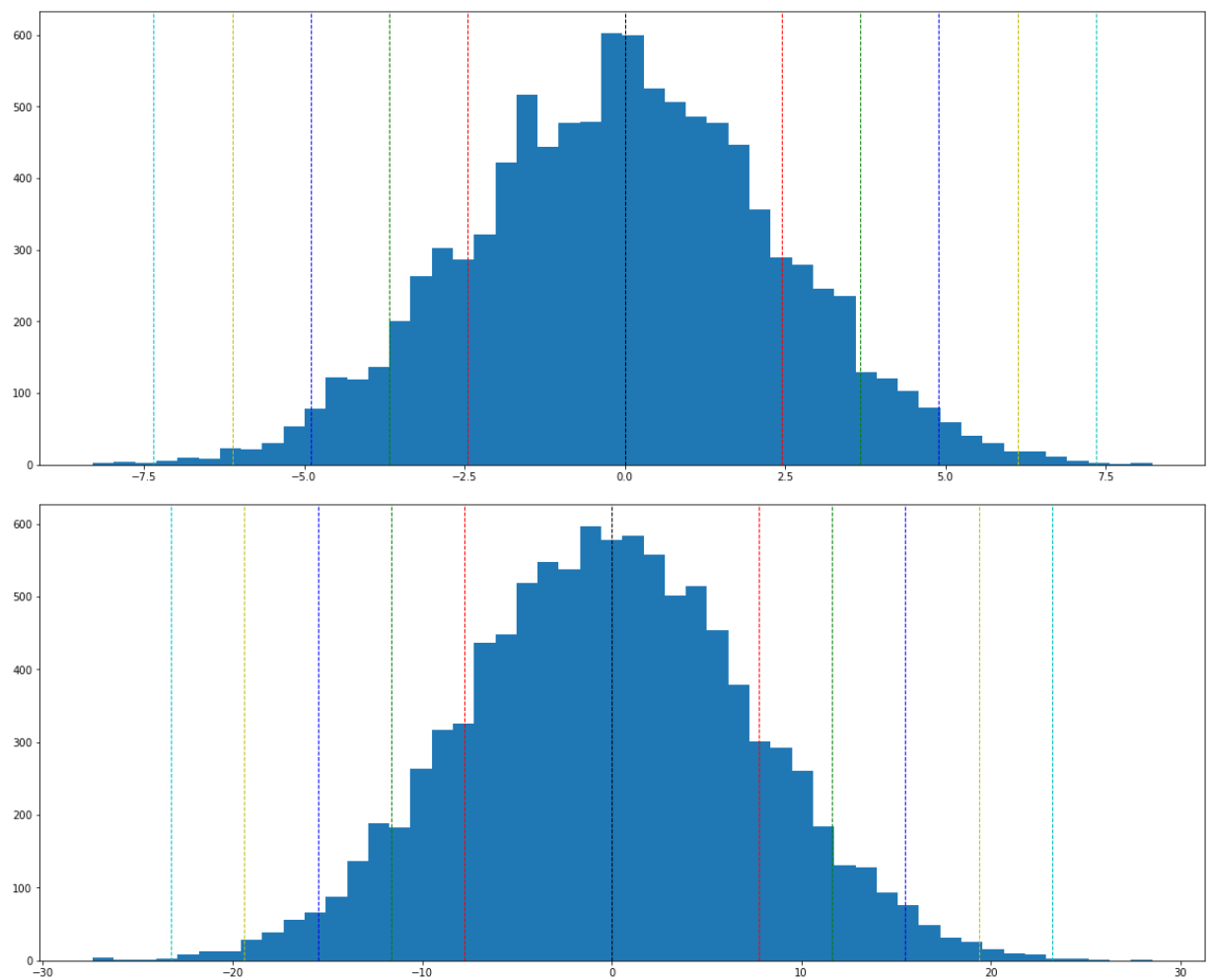
3

In [8]:

```
from matplotlib import pyplot as plt
import numpy as np
import scipy.stats as stats
plt.rcParams['figure.figsize'] = [20, 8]
nU=10300282
def geraDadosExclusao( N=1, f=lambda x:x**2 * 3/2, xI=-1, xS=+1, yMax=1 ):
    x = []
    while( len(x)<N ):
        xc = np.random.rand()*(xS-xI) + xI
        yv = yMax * np.random.rand()
        if yv<=f(xc):
            x.append(xc)
    return x

lamb=np.arange(1,3.1,.5)
np.random.seed(nU)
sx=(3/5)**.5
x0=0
r=[]
N=10000
colors=['r','g','b','y','c']
for i in [1,2,3,5,10,100]:
    np.random.seed(nU)
    dados = np.array(N*[0])
    for ii in range(i):
        dados =dados + np.array(geraDadosExclusao(N))
    plt.hist(dados,bins=50)
    plt.axvline(x0*i, color='k', linestyle='dashed', linewidth=1)
    for iii in lamb:
        #r.append(sum(dados<(i**.5*sx))-sum(dados<-(i**.5*sx)))
        r.append(sum(dados<((i**.5*iii*sx)+x0*i))-sum(dados<(-(i**.5*iii*sx)+x0*i)))
        plt.axvline(x0*i+(i**.5*iii*sx), color=colors[int(np.where(lamb==iii)[0])],
                    plt.axvline(x0*i-(i**.5*iii*sx), color=colors[int(np.where(lamb==iii)[0])],
        r.append(stats.skew(dados))
        r.append(stats.kurtosis(dados))
    plt.show()
for i in range(7):
    print(r[7*(i):7*(i+1)])
```





```
[5104, 10000, 10000, 10000, 10000, -0.010324939979243476, -1.7971028960814088]
[5595, 8599, 10000, 10000, 10000, 0.011092708611171988, -0.8545770823737464]
[7304, 8168, 9851, 10000, 10000, -0.002521158025260168, -0.5868643618603562]
[6675, 8793, 9539, 9966, 10000, 0.020999576764256973, -0.31948731284158294]
[6935, 8807, 9626, 9907, 9987, -0.0011416787128685267, -0.11796658855698317]
[6925, 8748, 9601, 9911, 9988, 0.01999137001966623, -0.07273905977721284]
[]
```

4

In [9]:

```
from matplotlib import pyplot as plt
import numpy as np
import scipy.stats as stats
plt.rcParams['figure.figsize'] = [20, 8]
nU=10300282

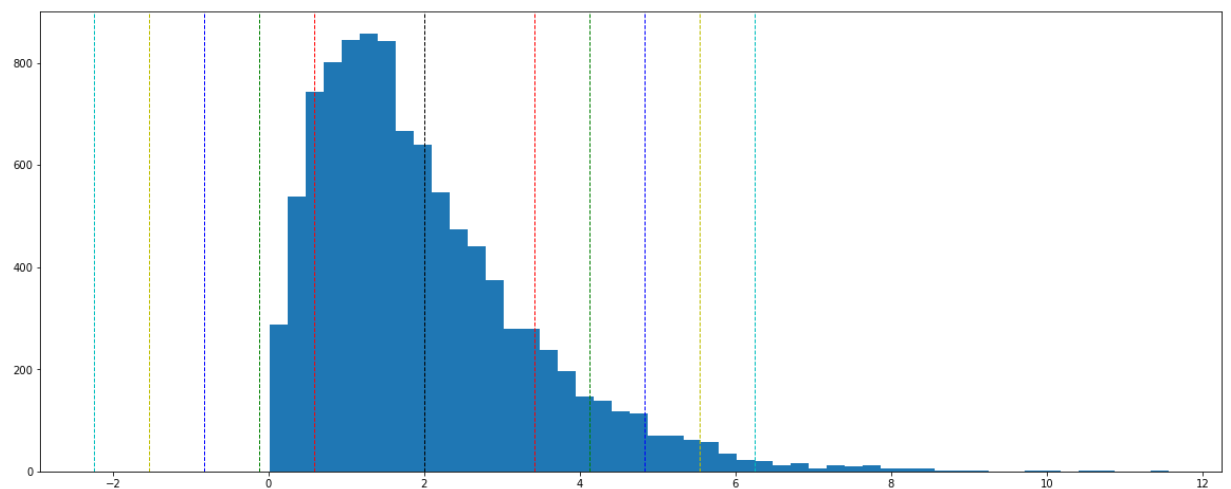
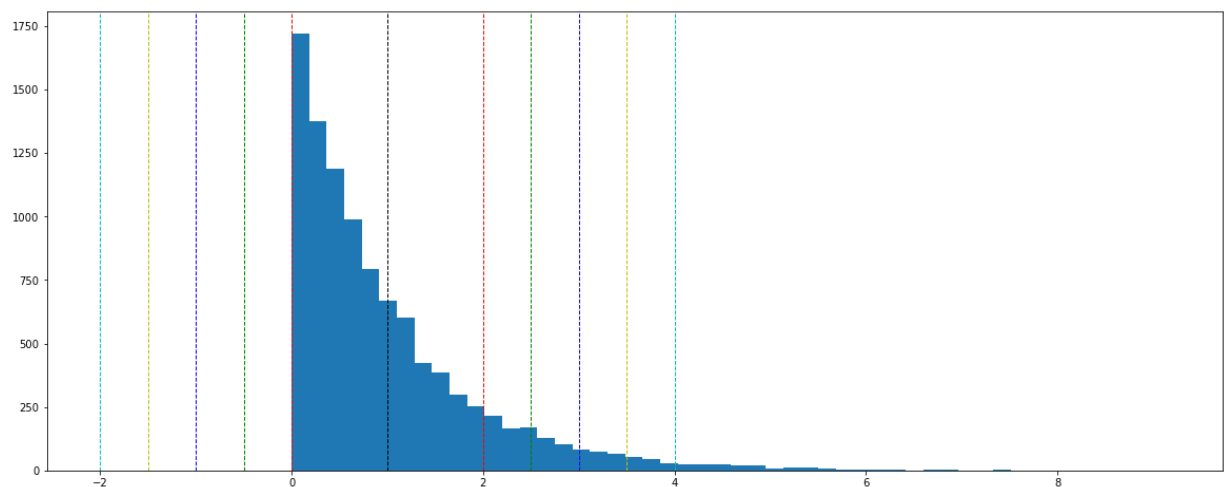
import numpy as np
def geraDadosInversao( N=1, invG=lambda g:-np.log(1-g) ):
    x=[]
    for i in range(N):
        g = np.random.rand()
        x.append( invG(g) )
    return x

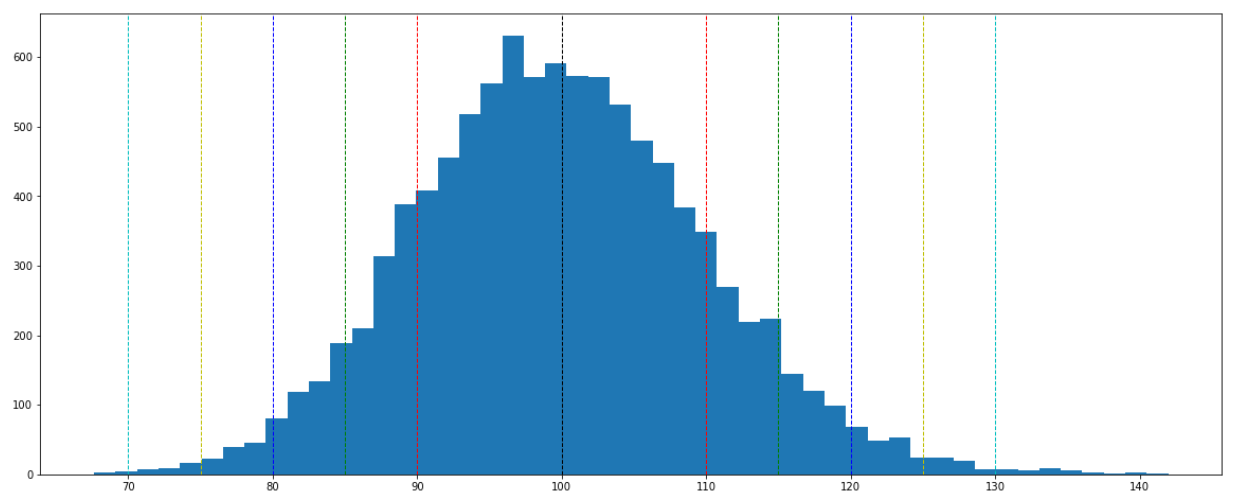
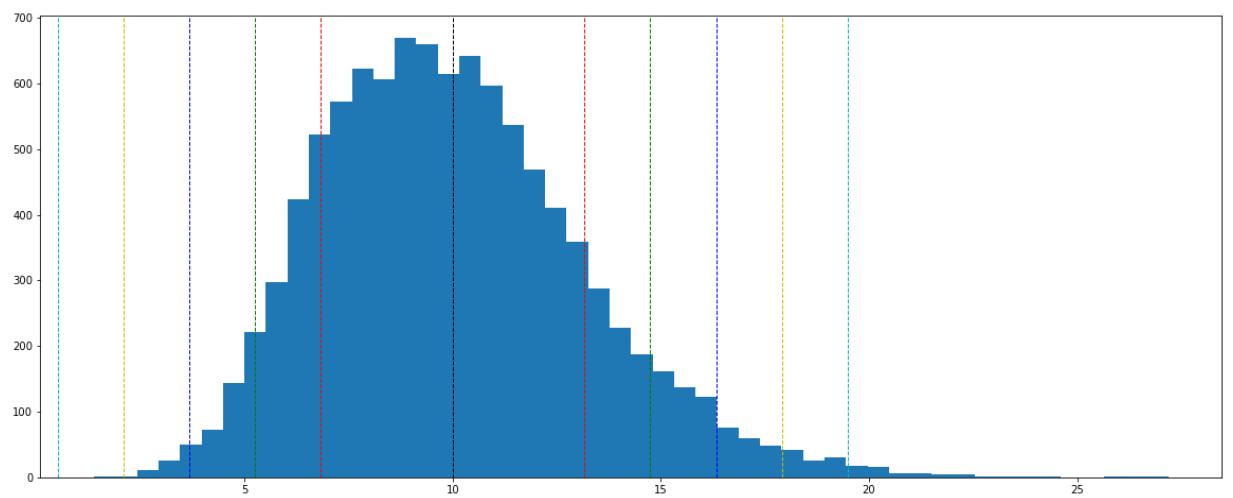
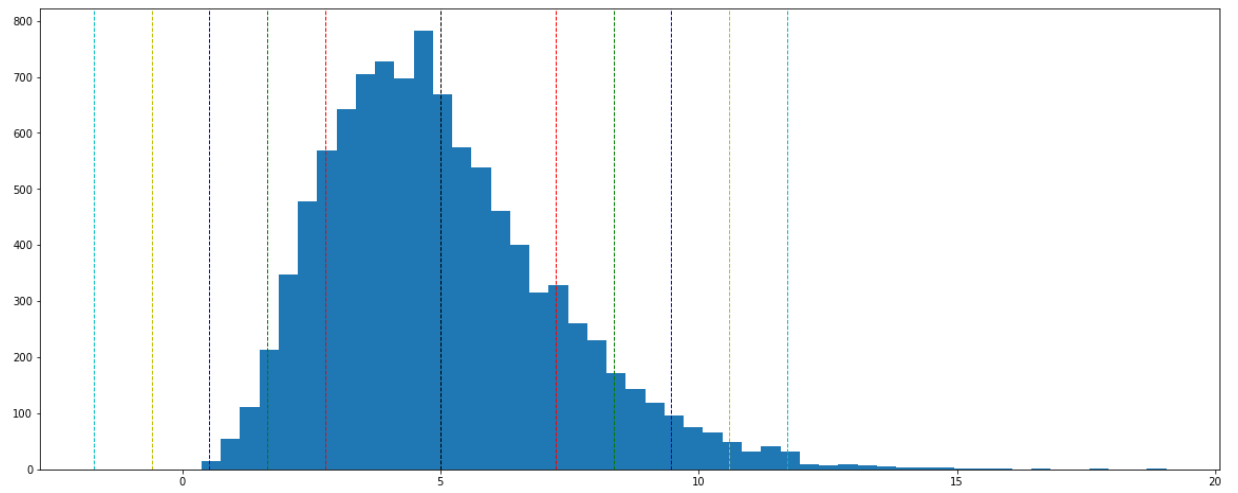
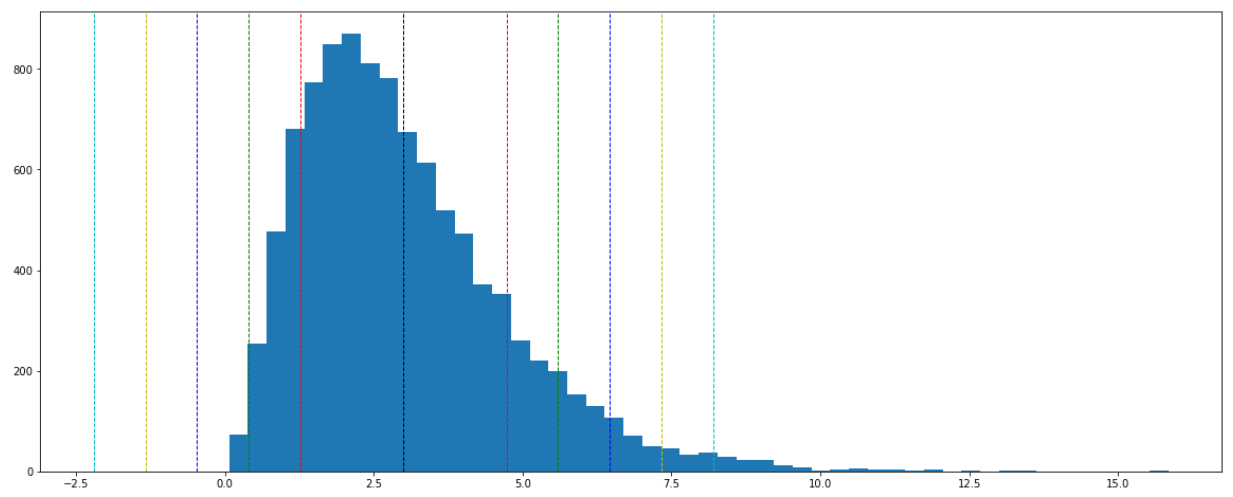
lamb=np.arange(1,3.1,.5)
np.random.seed(nU)
sx=1
x0=1
r=[]
N=10000
```

```

colors=['r','g','b','y','c']
for i in [1,2,3,5,10,100]:
    np.random.seed(nU)
    dados = np.array(N*[0])
    for ii in range(i):
        dados = dados + np.array(geraDadosInversao(N))
    plt.hist(dados,bins=50)
    plt.axvline(x0*i, color='k', linestyle='dashed', linewidth=1)
    for iii in lamb:
        #r.append(sum(dados<(i**.5*sx))-sum(dados<-(i**.5*sx)))
        r.append(sum(dados<((i**.5*iii*sx)+x0*i))-sum(dados<(-(i**.5*iii*sx)+x0*i)))
        plt.axvline(x0*i+(i**.5*iii*sx), color=colors[int(np.where(lamb==iii)[0])],
        plt.axvline(x0*i-(i**.5*iii*sx), color=colors[int(np.where(lamb==iii)[0])],
    r.append(stats.skew(dados))
    r.append(stats.kurtosis(dados))
    plt.show()
for i in range(7):
    print(r[7*(i):7*(i+1)])

```





[8679, 9184, 9500, 9701, 9823, 1.9117098891861914, 4.997958774290116]
 [7401, 9165, 9549, 9766, 9881, 1.342291973362106, 2.50711017688283]

```
[7178, 9081, 9565, 9763, 9873, 1.1733559414088135, 2.117885211036543]  
[6990, 8935, 9577, 9808, 9918, 0.8579992064759805, 1.0626732910525334]  
[6935, 8803, 9591, 9838, 9938, 0.5991295954396831, 0.5593978920099931]  
[6871, 8733, 9570, 9865, 9963, 0.21651050290259452, 0.14785699996802792]  
[]
```